

Relatório de Pesquisa: Arquitetura e Implementação de um Sistema de Reconhecimento de Gestos Baseado em TinyML para STM32F411RE

1. Introdução e Contextualização Arquitetural da Solução

A convergência entre a microeletrônica de baixo consumo e a inteligência artificial tem catalisado o desenvolvimento de interfaces homem-máquina imersivas, com particular destaque para os dispositivos vestíveis (*wearables*).¹ No âmbito das tecnologias de assistência e do reconhecimento da cinemática humana, o desenvolvimento de uma *smart glove* (luva inteligente) capaz de identificar um dicionário restrito de gestos — englobando letras estáticas do alfabeto gestual e movimentos dinâmicos simples — representa um desafio de engenharia multidisciplinar. A premissa central deste projeto reside na validação de uma arquitetura de hardware e software que integre algoritmos de *Machine Learning* embarcado (TinyML) diretamente no limite da rede (*edge computing*), fundindo dados multimodais provenientes de sensores de forma eficiente e determinística.

O hardware selecionado como unidade central de processamento é o microcontrolador STMicroelectronics STM32F411RE. Integrado na família de processadores de alto desempenho, este dispositivo é impulsionado por um núcleo ARM Cortex-M4F operando a uma frequência máxima de 100 MHz, complementado por uma Unidade de Ponto Flutuante (FPU) e instruções de Processamento Digital de Sinal (DSP).⁴ O mapeamento de memória deste microcontrolador estipula um limite físico de 512 KB de memória Flash para o armazenamento não-volátil do firmware e dos pesos da rede neuronal, e, mais criticamente, 128 KB de memória SRAM estática para a alocação do *heap*, variáveis globais, pilhas de execução (*stack*) e *buffers* de tensores durante a inferência.⁵ Esta limitação de 128 KB de SRAM ditará rigorosamente as escolhas arquiteturais em todas as fases do desenvolvimento, desde a amostragem até à topologia do modelo de inferência.⁶

A matriz de captação sensorial assenta numa fusão de dados cinemáticos e biomecânicos. Para a captura da orientação espacial, aceleração linear e velocidade angular da mão, a arquitetura incorpora um módulo de Unidade de Medida Inercial (IMU) com seis graus de liberdade (6-DoF), especificamente o MPU6050, comunicando através do barramento I2C.⁷ Simultaneamente, para quantificar o arco de flexão falangeal, a luva é equipada com um conjunto de três a cinco sensores de flexão analógicos (*flex sensors*), cuja variação resistiva é

lida pelo Conversor Analógico-Digital (ADC) interno de 12 bits do STM32F411RE.⁸

Dado que o microcontrolador STM32F411RE não possui periféricos nativos de radiofrequência, a exfiltração dos dados classificados para o exterior depende de um módulo de comunicação secundário. O projeto contempla o uso de um módulo Bluetooth Clássico HC-05 ou um módulo Wi-Fi ESP8266, ambos acoplados fisicamente à interface *Universal Synchronous and Asynchronous Receiver-Transmitter* (UART) do microcontrolador.¹⁰ A missão final da arquitetura culmina num terminal de apresentação de dados: um computador local executando um *dashboard* web que escuta passivamente a porta serial (ou socket TCP/UDP) e atualiza, em tempo real, a interface gráfica com a tradução do gesto capturado, minimizando a latência global do sistema.³

A conceção desta topologia exige a resolução profunda de vários estrangulamentos clássicos em sistemas embarcados. Torna-se imperativo selecionar o *framework* TinyML que melhor otimiza o uso da restrita SRAM; desenhar um *pipeline* de amostragem de dados que não bloqueie a Unidade Central de Processamento (CPU); aplicar filtros digitais robustos para o ruído vibracional do MPU6050 e o fenómeno de *bouncing* elétrico dos sensores analógicos; e, finalmente, estabelecer uma *stack* de apresentação web que suporte o fluxo assíncrono de dados sem induzir bloqueios (*deadlocks*) de visualização.

2. Análise e Seleção de Frameworks TinyML

A implantação de redes neuronais em microcontroladores com constrangimentos severos de memória impulsionou a criação de ecossistemas de software especializados, genericamente apelidados de *frameworks* TinyML.¹³ A escolha da plataforma correta para o hardware STM32F411RE ditará não apenas a exatidão do modelo de reconhecimento de gestos, mas principalmente a viabilidade técnica de o manter em execução simultânea com as rotinas de amostragem de sensores dentro da margem de 128 KB de SRAM.⁶ A análise foca-se nas três plataformas mais proeminentes da indústria: TensorFlow Lite for Microcontrollers (TFLite Micro), Edge Impulse e a solução nativa da STMicroelectronics, X-CUBE-AI.

2.1. TensorFlow Lite for Microcontrollers (TFLite Micro)

O TensorFlow Lite for Microcontrollers é a biblioteca fundacional de código aberto desenvolvida pela Google, concebida explicitamente para executar operações de inferência em processadores de 32 bits com escassos recursos.¹⁶ A sua filosofia de design baseia-se na portabilidade extrema, sendo o código escrito em C++11 e desprovido de dependências de alocação dinâmica de memória, sistemas operativos ou bibliotecas padrão de C/C++ pesadas.¹⁷

A mecânica do TFLite Micro exige que o modelo seja previamente treinado num ambiente *desktop* ou *cloud* padrão (utilizando TensorFlow/Keras) e, posteriormente, convertido num ficheiro FlatBuffer (.tflite) otimizado.¹⁶ Este ficheiro é então transformado num *array* constante

de C, que é compilado diretamente no firmware do STM32.¹⁶ Durante a inicialização do interpretador no microcontrolador, o programador é obrigado a instanciar uma arquitetura de alocação de memória estática conhecida como *Arena Allocator*.¹⁷ Esta área contígua de memória é o espaço exclusivo onde o TFLite Micro irá alocar os tensores de entrada, tensores de saída e os *buffers* temporários (*scratch buffers*) exigidos pelas ativações das camadas intermédias da rede neuronal.

O desafio prático mais severo da utilização do TFLite Micro em formato *bare-metal* no STM32F411RE é o empirismo associado à configuração deste *Arena Size*. O framework não possui um mecanismo de cálculo preditivo do *heap* necessário antes da execução.¹⁵ O engenheiro é forçado a declarar um tamanho arbitrário de RAM e tentar carregar o modelo; se a alocação falhar devido à complexidade matricial das operações de convolução ou ligação densa, o interpretador devolve um erro em tempo de execução (*runtime error*), forçando a recompilação interativa do código com incrementos graduais da memória alocada.¹⁵ Além disso, o TFLite Micro opera como um interpretador genérico. Cada nó do grafo computacional é analisado sequencialmente e a função do kernel correspondente é invocada dinamicamente, o que introduz uma sobrecarga (*overhead*) de tempo de inferência e de consumo de memória RAM significativamente maior comparativamente a abordagens que compilam a rede antecipadamente.¹⁵ Esta carência de otimização estrita orientada ao hardware inviabiliza a sua recomendação como ferramenta primária num dispositivo com limites tão estritos de SRAM.

2.2. Edge Impulse e o Compilador EON

O Edge Impulse emerge como uma plataforma de Desenvolvimento Integrado (IDE) *end-to-end* baseada em infraestrutura *Cloud*, desenhada para abstrair a intrincada matemática da aquisição, processamento e modelagem de dados sensoriais em dispositivos IoT.¹³ A vasta disparidade entre o TFLite Micro e o Edge Impulse manifesta-se em duas vertentes críticas para o reconhecimento de gestos: a extração de características (Processamento Digital de Sinal - DSP) e o compilador patenteado EON (*Edge Optimized Neural*).

A análise de dados inerciais em bruto (Aceleração e Velocidade Angular) através de redes neuronais densas costuma ser altamente ineficiente e propensa a *overfitting*. O Edge Impulse integra nativamente blocos de processamento que executam a Análise Espectral dos sinais (como a Transformada Rápida de Fourier - FFT e Densidade Espectral de Potência - PSD) sobre as janelas de amostragem antes de injetar os tensores na rede.¹⁹ Ao transformar o domínio temporal em domínio da frequência, as assinaturas fundamentais de gestos como uma flexão ("Flex") ou um soco ("Punch") tornam-se lineares e altamente separáveis, permitindo reduzir drasticamente o número de parâmetros da rede neuronal subsequente.²⁰

A inovação mais profunda do Edge Impulse para microcontroladores é o compilador EON. Contrariamente à abordagem interpretativa do TFLite Micro, o compilador EON analisa a arquitetura da rede neuronal treinada e gera código fonte em C/C++ puro, construindo

estruturas de dados estáticas sem a presença de um interpretador subjacente.¹³ Estudos comparativos documentam que a eliminação deste *overhead* resulta numa execução que consome entre 25% a 55% menos SRAM e até 35% menos memória Flash, mantendo exatamente a mesma precisão de inferência matemática.¹³ Numa placa STM32 Cortex-M4F, esta compressão revela-se fundamental para assegurar que a execução da rede convive harmoniosamente com os pesados *buffers* de receção de dados dos sensores de flexão e do IMU.¹³ Apesar da sua excelência arquitetural, a dependência do Edge Impulse como solução autossuficiente pode limitar personalizações profundas de baixo nível na alocação de registos DMA ou na manipulação de prioridades de interrupção (NVIC) diretamente no IDE base da STMicroelectronics, tornando-se imperativa a sua integração num fluxo híbrido.¹³

2.3. STM32Cube.AI (X-CUBE-AI)

A resposta corporativa da STMicroelectronics aos constrangimentos do TinyML corporizou-se no X-CUBE-AI, um pacote de expansão nativo perfeitamente incrustado no ecossistema STM32CubeMX.¹⁸ Diferenciando-se das ferramentas de alto nível, o X-CUBE-AI atua como um conversor e otimizador profundo: aceita grafos computacionais pré-treinados exportados dos pacotes Keras, PyTorch, ONNX ou TFLite, e transcreve a topologia matemática para uma biblioteca dinâmica em linguagem C, intrinsecamente afinada para a Unidade Lógica Aritmética (ALU) e FPU específicas do microcontrolador alvo.²²

O trunfo absoluto do X-CUBE-AI sobre as alternativas reside no determinismo estrutural da memória. Durante o processo de conversão e compilação do modelo, a interface gráfica avalia a arquitetura, simula o grafo de inferência e reporta metricamente o consumo milimétrico exato da Flash (pesos e viés) e da RAM (tensores de ativação iterativos) necessários.¹⁵ Este reporte analítico elimina completamente as aproximações empíricas do *Arena Size* exigidas pelo TFLite Micro.¹⁵ Adicionalmente, a ferramenta da STMicroelectronics oferece estratégias de quantização em INT8 (números inteiros de 8 bits) em vez de ponto flutuante de 32 bits (Float32). O mapeamento de *Floats* para inteiros corrompe apenas uma fração marginal (cerca de 1% a 3%) da exatidão preditiva global do modelo, mas fragmenta as exigências de memória em quase quatro vezes, ao mesmo tempo que reduz exponencialmente os tempos de inferência (que em placas da família F4 podem mergulhar para menos de 10 milissegundos).²⁴

2.4. Resumo e Seleção Final da Arquitetura de Software

A tabela abaixo sumariza as características das três soluções:

Característica de Arquitetura	TFLite Micro	Edge Impulse (Compilador EON)	X-CUBE-AI (STMicroelectronics)

Paradigma de Execução	Interpretador em tempo real	Código C++ gerado e nativo	Biblioteca C nativa e otimizada
Mecanismo de Alocação de Memória	Dinâmico e empírico (<i>Arena Allocator</i>)	Estático e otimizado automaticamente	Totalmente determinístico, validado pré-compilação
Eficiência (SRAM/Flash)	Elevada, gera sobrecarga de processamento	Superior (poupa entre 25% e 55% de RAM)	Máxima otimização nativa do hardware STM32
Integração de Processamento de Sinal	Nula (requer código e bibliotecas de terceiros)	DSP avançado nativo (FFT, Filtros FIR)	Foco exclusivo na inferência; exige tratamento de sinal em C

Recomendação: A arquitetura ótima e de maior robustez para a topologia da *smart glove* fundamenta-se num **paradigma híbrido de cooperação entre o Edge Impulse e o X-CUBE-AI**.

A engenharia de concepção dita que a fase de recolha empírica dos dados em bruto dos sensores, bem como o processo iterativo de *Labeling* (rotulagem de dados) e o cálculo de características DSP (Análise Espectral de Frequências) devem ser operados utilizando o *Studio* da nuvem do **Edge Impulse**, beneficiando da sua incomparável visualização gráfica dos eixos.¹⁸ Após a fase de validação e treino estrutural da rede, o programador deve transitar para as opções de desdobramento e instruir o Edge Impulse a exportar o ambiente como um modelo compatível com o motor CMSIS-PACK do **STM32Cube.AI**.¹⁸ Esta integração injeta perfeitamente a matemática no STM32CubeIDE, fundindo o melhor processamento de sinal existente na web com o compilador C nativo mais performante para processadores Cortex-M, assegurando o máximo respeito pelas fronteiras dos 128 KB de RAM disponíveis.

3. Fundamentações Biomecânicas e Frequência de Amostragem

O desígnio de um modelo de *Machine Learning* estritamente condicionado aos pressupostos

do TinyML baseia-se numa correlação inflexível: a dimensão da janela temporal de amostragem determina diretamente o número de amostras capturadas e, consecutivamente, a largura dos tensores de entrada da rede neuronal. Uma rede excessivamente densa consome recursos proibitivos. Por conseguinte, a definição científica da frequência de amostragem ideal (F_s) é o pilar que garante a minimização de tráfego de dados, sem omitir eventos cruciais da dinâmica física.

3.1. Análise Frequencial da Cinemática da Mão Humana

Estudos biomecânicos e de análise de atividades quotidianas demonstram que as interações esqueléticas e articulares do ser humano ocorrem predominantemente num espectro de frequências de banda muito baixa. O impacto das taxas de amostragem de Unidades de Medida Inercial na precisão dos algoritmos de fusão sensorial e de classificação exhibe uma curva de degradação bem documentada na literatura médica e técnica.²⁸

Foi provado experimentalmente que frequências de captação que ultrapassam largamente as capacidades biológicas (acima dos 100 Hz) não só são supérfluas como provocam um impacto deletério na inferência matemática.²⁹ A altas frequências, os tensores do acelerómetro começam a captar e a focar-se no ruído ambiente e ressonâncias microscópicas do sensor, introduzindo distorções no modelo ao invés de representarem as reais atualizações vetoriais angulares e de inércia produzidas pela mão humana.²⁹

No polo oposto, a redução drástica das frequências de observação revela que taxas perto dos 10 Hz preservam uma excelente precisão (*accuracy*) na monitorização global de posturas estáticas, repouso, ou locomoção de baixa cadência.³⁰ Todavia, se a frequência descender até ao patamar exiguo de 1 Hz, a exatidão decai categoricamente para movimentos interativos repetitivos ou espasmódicos (como gesticular ativamente uma emoção, varrimentos balísticos da mão ou tremores de punho característicos da linguagem gestual avançada).³⁰

3.2. Teorema de Nyquist e Seleção do Ótimo Local

De acordo com o princípio matemático de Nyquist-Shannon, para reconstruir com exatidão um sinal analógico e garantir a ausência do fenómeno de subamostragem (*aliasing*), a frequência de amostragem necessita de ser ligeiramente superior ao dobro da frequência máxima observável da fonte original. Considerando que transições cinemáticas dinâmicas da mão (súbitas contrações falangeais para simular um aperto) raramente excedem um período de 15 a 20 Hz, a amostra limite orbita nas vizinhanças dos 40 Hz a 50 Hz.³²

A parametrização iterativa efetuada por diversas metodologias de pesquisa focadas em soluções TinyML suportadas em baterias solidifica que **a fixação do ritmo de captação a 50 Hz (um intervalo exato de 20 milissegundos por amostra) assume-se como o ótimo global** para a aquisição coordenada de IMU e acelerometria aplicada a membros superiores.²⁷ Uma amostragem síncrona a 50 Hz retém 100% da identidade transitória do movimento perante o

algoritmo sem expandir a matriz dimensional aos limiares destrutivos da SRAM no MCU e viabiliza a execução suave e sequencial no tempo real do *loop* principal.

4. Pipeline de Recolha de Dados (Amostragem Multimodal e Sincronização)

A viabilidade estatística de um *dataset* de inferência fundamenta-se num critério inegociável de engenharia: cada amostra temporal proveniente da varredura paralela dos 6 canais inerciais do IMU (MPU6050 via I2C) e dos 5 canais resistivos de flexão falangeal via Conversor Analógico-Digital (ADC) deve estar rigidamente sincronizada num *timestamp* exato.⁸

A captação destes 11 canais isolados através da arquitetura de *polling* sequencial (um método vulgarmente utilizado num ambiente amador com o ciclo `while(1)` da Arduino IDE) introduz desvios latentes drásticos.³⁵ Quando a CPU solicita um barramento, aguarda pela reposta inercial e subsequentemente manda ler as portas ADC em série, a divergência temporal destrói a coesão cronológica do dado. Adicionalmente, o *polling* acorrenta e asfixia a capacidade da Unidade de Ponto Flutuante (FPU) processar os cálculos densos da rede neuronal local (inferência), uma vez que a CPU se perde na gestão estúpida e micro-bloqueante dos sensores.³⁵ O paradigma arquitetural deve basear-se imperativamente numa estrutura governada por eventos (*Event-Driven*) orquestrada por temporizadores de hardware independentes e processos de Acesso Direto à Memória (DMA).

4.1. Arquitetura Não-Bloqueante Baseada em Eventos e DMA

A transição de uma arquitetura centralizada na CPU para uma malha distribuída baseada em DMA no microcontrolador STM32F411RE executa as rotinas sensoriais de forma invisível. A estrutura opera do seguinte modo lógico:

1. **Relógio Mestre (Timer Trigger):** O periférico do Temporizador Básico de Hardware do STM32 (como o TIM3) é configurado para funcionar ininterruptamente e estourar (*overflow*) exaustivamente a cada 20 milissegundos, gerando uma taxa contínua e imutável de 50 Hz.³⁸ Este evento de estouro é configurado internamente para despachar um pulso elétrico designado *Trigger Output* (TRGO).
2. **Cascata ADC Sequencial:** O TRGO interliga-se diretamente via matriz de silício e excita o periférico ADC1. O conversor deve ser iniciado em modo de "Scan Circular Multi-Channel".³⁶ Ao sentir o pulso de 50 Hz, o ADC procede de forma atômica e autossuficiente à leitura veloz dos 5 canais lógicos dos sensores de flexão (ex: Pinos PA0 a PA4). Terminada a varredura completa analógica, o controlador ADC ativa um sinal informando o controlador DMA de que a captação terminou.
3. **Transporte DMA e Interrupção:** O canal da *Direct Memory Access* transfere, sem interrupção nem despertar do processador central, as amostras digitais acabadas de compor dos registos estáticos do ADC (ADC_DR) diretamente para um vetor (*array*) formatado alojado na SRAM da placa.³⁶

4. **Tráfego I2C em Paralelo:** Em simultaneidade harmoniosa, os seis vetores cinemáticos procedentes do registo do MPU6050 (Aceleração XYZ e Giroscópio XYZ) são solicitados.⁸ Esta leitura não obedece a um clássico chamamento síncrono. Encarrega-se de uma instrução não obstrutiva e de acesso de memória cruzado como a HAL_I2C_Mem_Read_DMA, ou de um alerta baseado numa Interrupção Externa (EXTI) ligada ao pino de interrupção embutido no componente (INT Pin do MPU6050), assegurando que o barramento é processado eficientemente sem desviar contagens computacionais.⁴²

4.2. Estratégia de "Double Buffering" para Transmissão Sustentável

A formatação e extração volumétrica das 11 variáveis captadas num compasso apertado de 50 Hz resultam na criação incessante de um pacote numérico denso que importa transmitir a uma interface de registo para treino posterior na nuvem de TinyML. A conversão digital do sinal processa-se largamente a uma velocidade incrivelmente maior do que a lenta emanação binária da porta Serial-UART. A sobreposição assíncrona destes ritmos vai inevitavelmente instigar um choque letal: a recolha vai sobrepor dados cruciais novos a posições de SRAM em que o módulo USART ainda não tenha terminado a transcrição e despacho byte a byte de dados prévios.⁴⁵

A contramedida científica e arquitetural denomina-se "Buffer Duplo" ou *Ping-Pong Buffering*.⁴⁵ Um espaço na RAM é compartimentado em dois vetores multidimensionais. O hardware (DMA) derrama a sua avalanche de dados recém-lidos ativamente no primeiro recipiente (o *buffer* passivo). Quando esse tanque satura-se à exatidão temporal projetada (por exemplo, contendo as amostras resultantes da janela deslizante pré-classificação), o microcontrolador executa inversão lógica atômica dos endereços estáticos aos quais os ponteiros referenciam. O hardware muda e começa a encher imediatamente o segundo recipiente virginal, deixando em paz a CPU do MCU para invocar sem interrupções os rotineiros cálculos lógicos, de inferência neuronal, ou compilações de *printf()* no primeiro recipiente estabilizado e imutável.⁴⁵

A estruturação arquitetural recomendável do firmware na linguagem C ou C++ expõe-se do seguinte modo de modelagem para a HAL da STM32:

C

```
// Dimensão topológica das estruturas estáticas
#define AMOSTRAS_POR_PACOTE 50 // Correspondente a janela temporal base (1 segundo de amostragem a 50 Hz)
#define NUM_CANAL_FLEX_ADC 5
#define NUM_CANAL_IMU 6
```

```

// Objeto empacotado individual
typedef struct {
    int16_t imu_data[NUM_CANAL_IMU];
    uint16_t flex_data;
} Amostra_Cinematica_T;

// Memórias matriz independentes
Amostra_Cinematica_T buffer_Primary;
Amostra_Cinematica_T buffer_Secundario;

// Definições voláteis de rastreamento para troca
Amostra_Cinematica_T* buffer_Preenchimento = buffer_Primary;
Amostra_Cinematica_T* buffer_Transmissao = buffer_Secundario;

volatile uint8_t processar_flag_pronto = 0;
volatile uint16_t ind_amostra = 0;

/* Callback Atômica da Camada HAL ativada exaustivamente apenas
quando os buffers do DMA alcançam o limite projetado de Scan Completo */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {

    // Obtenção síncrona adjacente via MPU do acelerômetro em barramento DMA ou Global I2C estrito
    buffer_Preenchimento[ind_amostra].imu_data = imu_regs.Accel_X;
    // [...] eixos Y, Z e Gyro

    // Incremento iterativo temporal
    ind_amostra++;

    // Interservação (Ping-Pong Switch) se a dimensão matriz completou o limite da amostragem paralela
    if (ind_amostra >= AMOSTRAS_POR_PACOTE) {
        if (buffer_Preenchimento == buffer_Primary) {
            buffer_Preenchimento = buffer_Secundario;
            buffer_Transmissao = buffer_Primary;
        } else {
            buffer_Preenchimento = buffer_Primary;
            buffer_Transmissao = buffer_Secundario;
        }
        ind_amostra = 0;

        // Elevação de flag para desbloquear operações pesadas de formatação em plano de fundo no loop
        infinito (while)
        processar_flag_pronto = 1;
    }
}

```

```
}
```

Ao encapsular o transporte em plano de fundo no interruptor passivo gerado via HAL_ADC_ConvCpltCallback, garante-se que a espinha dorsal de execução temporal opera livre e que os sensores fornecem resultados unificados sem atraso de interposição de hardware.

No *loop* principal bloqueante (`while(1)`), o software desvia de processamento passivo de baixo-consumo e avalia se o flag ativou.³⁵ Em afirmativa, aciona-se a tradução pesada e processamento do pacote total transmitindo a janela pela serial ligada ao Módulo PC usando transferências que se adaptam eficientemente a altas velocidades na via UART:

C

```
int main(void) {
    /* Inicializações cruciais padrão STM32 */
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_I2C1_Init();
    MX_USART2_UART_Init();
    MX_ADC1_Init();
    MX_TIM3_Init();

    // O ADC prosseguirá varrendo a matriz DMA na ligação lógica da estrutura em uso
    HAL_ADC_Start_DMA(&hadc1, (uint32_t*)buffer_Preenchimento->flex_data,
    NUM_CANAL_FLEX_ADC);

    // O trigger do temporizador inicia a engrenagem cíclica a 50Hz baseada em eventos
    HAL_TIM_Base_Start(&htim3);

    char buffer_texto_transmitir;

    while (1) {
        if (processar_flag_pronto) {
            processar_flag_pronto = 0; // Desarme de segurança e tranca lógica

            // Loop extensivo para renderizar amostra a amostra dos dados estáticos retidos e protegidos
            for (uint16_t x = 0; x < AMOSTRAS_POR_PACOTE; x++) {
```

```

        sprintf(buffer_texto_transmitir, "%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d\n",
            buffer_Transmissao[x].imu_data, buffer_Transmissao[x].imu_data,
            buffer_Transmissao[x].imu_data, buffer_Transmissao[x].imu_data,
            buffer_Transmissao[x].imu_data, buffer_Transmissao[x].imu_data,
            buffer_Transmissao[x].flex_data, buffer_Transmissao[x].flex_data,
            buffer_Transmissao[x].flex_data, buffer_Transmissao[x].flex_data,
            buffer_Transmissao[x].flex_data);

        // Emissão com API não obstrutiva de UART-DMA e alta cadência em Bauds (ex: 115200)
        HAL_UART_Transmit_DMA(&huart2, (uint8_t*)buffer_texto_transmitir,
strlen(buffer_texto_transmitir));
    }
}
}
}
}

```

Nota Arquitetural Complementar: Importa referir que a UART1 ou a USART2 do STM32F411RE partilham compatibilidades singulares. Numa placa de desenvolvimento tipo *Nucleo*, os pinos de abstração PA2 e PA3 encostam diretamente na arquitetura matricial do ST-LINK V2 embarcado. Isto converte internamente a malha eletrónica e imita instantaneamente uma Com-Port de ponte Serial para USB no sistema central Windows/Linux.⁴⁶ Consequentemente, para a recolha estrita dos *datasets* iniciais, pode-se usar esta conexão USB robusta eliminando latências externas imprevisíveis do modelo *Bluetooth* para obter um ficheiro limpo.

4.3. Interface Python de Arquivo Central (CSV)

Com as cadências do microcontrolador ajustadas e formatadas com separadores lógicos ("comma-separated values" num *string frame* encerrando pela quebra virtual do carácter de escape \n), a base do PC assume agora a premissa de um recetor limpo.⁴⁹ Um script interpretativo em linguagem Python atua com precisão, instanciando chamamentos paralelos que observam assincronamente as portas via módulo genérico pyserial.⁴⁹

O código deve abrir a conexão limitando perdas passivas. É essencial usar a função codificada do bloco de instrução base do SO que trava preventivamente ciclos excessivos sem tráfego de chegada (ser.in_waiting) para gerir latências da própria cache base do USB.⁴⁹

Python

```

import serial
import csv

```

```

import sys

# Configurações dinâmicas de interface e nome de ficheiro-alvo do gesto local
PORTA_COM_STM32 = 'COM3' # Exemplo adaptável à COM mapeada do ST-Link (gerenciador de
dispositivos)
TAXA_BAUD = 115200
ARQUIVO_DATASET = "dataset_gesto_L.csv"

try:
    conexao_serial = serial.Serial(PORTA_COM_STM32, TAXA_BAUD, timeout=1)
except serial.SerialException as erro:
    print(f"Erro fatal na ponte USB-Serial: {erro}")
    sys.exit()

print(f" - Escuta engatada à {TAXA_BAUD} bps. Utilize (CTRL+C) para interromper captação de frames.")

with open(ARQUIVO_DATASET, mode='a', newline="") as ficheiro_aberto:
    gerente_csv = csv.writer(ficheiro_aberto)
    # Geração dos cabeçalhos vetoriais de importação exigidos pelos módulos web tipo Edge Impulse.
    gerente_csv.writerow()

    try:
        while True:
            # Trava lógica bloqueando a iteração circular de processamento desnecessário Python até
            chegarem novos bits.
            if conexao_serial.in_waiting > 0:
                pacote_bruto = conexao_serial.readline().decode('utf-8').strip()
                trama_vetorial = pacote_bruto.split(',')

                # Barreira de validação estrutural a prevenir lixo residual ou corrupção cruzada em pacotes
                descartados.
                if len(trama_vetorial) == 11:
                    gerente_csv.writerow(trama_vetorial)
                    # Opcional log print para validação visual (suprimir para evitar congestionamento UI terminal
                    a ritmos elevados)
                    # print(trama_vetorial)

    except KeyboardInterrupt:
        print("\n\n - Gravação suspensa com segurança pelo operador.")
        conexao_serial.close()

```

Esta robustez algorítmica forja um ambiente seguro de extração analítica e processa a passagem de pacotes de formato em bruto (.csv) prontos e compatíveis a injetar nos

laboratórios focados na inferência de *Machine Learning* na *Cloud*, como atesta a biblioteca analítica estrita partilhada entre dispositivos.²⁰

5. Mitigação e Rejeição de Anomalias Físicas (Condicionamento de Sinal)

A discrepância crassa entre a formatação simulada em ambiente estéril no PC e o comportamento matemático do hardware embarcado encontra-se quase sempre na imperfeição entrópica da física aplicada. Sinal sujo aniquila redes neurais pequenas, degradando a confiabilidade das respostas e elevando a predição para resultados estatisticamente flutuantes e aleatórios.²¹ A mitigação rigorosa destas anomalias na nascente dos barramentos (Módulo MPU e resistores flexíveis) tem de ser atacada com engenharia computacional pragmática antes do encapsulamento em modelos TinyML.⁷

5.1. Rejeição Estrutural no MPU6050

As unidades baseadas em Acelerometria e Giroscópio integradas no MEMS do *chip* MPU6050 revelam comportamentos de extrema oscilação na curva dos sinais emitidos em virtude de componentes mecânicos envolventes, vibrações sonoras parasitas e perturbações eletromagnéticas que provocam tremores de alta frequência nas leituras dos vetores absolutos.⁵² Se deixadas em bruto, estas espículas flutuantes e espúrias vão sobrecarregar a janela de extração de características (FFT) da rede neural com padrões fantasmas indissociáveis do ruído branco do sensor.²¹

Num ecossistema dominado por uma arquitetura onde se escrutina ao máximo os recursos do CPU (para reservar SRAM para as redes do Edge Impulse / Cube.AI), seria fatídico carregar um programa baseado em *software* C focado em eliminar o ruído na totalidade dos 6 vetores de ponto flutuante.⁵⁵ A solução cientificamente preconizada reside no aproveitamento transversal do filtro de *hardware* cravado na malha de silício intrínseca do próprio integrado I2C MPU6050: o **Digital Low-Pass Filter** (DLPF).⁷

A manipulação da porta analógica de filtragem realiza-se diretamente via intervenção lógica nos canais de I2C apontados para a *Configuração de Registo Principal* (O registo hexadecimal universal de número 0x1A).⁷ Uma vez que os tremores interativos e de alta repetição nas locomoções manuais humanas situam-se firmemente em valores de ressonância limitados em torno das frequências inferiores a 15 Hz, a seleção de uma máscara binária limitante para o DLPF no limiar dos cerca de 20 Hz esmaga e cega mecanicamente todo o conteúdo vibracional dissonante superior, entregando valores analógicos profundamente suavizados no instante anterior à ação central de discretização efetuada pelos conversores de 16-bits da placa do sensor.⁷ O processo evita absolutamente ciclos obstrutivos nulos processados pela arquitetura M4F.

Múltiplas literaturas académicas e documentações originadas em fóruns debruçam-se em abordagens focadas num modelo denso como os *Filtros de Kalman* acoplados com as variantes complementares (exemplo matemático expressamente escrito na forma

$\theta_t = \alpha \times (\theta_{t-1} + \omega \times \Delta t) + (1 - \alpha) \times a$) com a pretensão de fundir as duas métricas instáveis originadas pelo acelerómetro flutuante e giroscópio enviesado em desvios (*drift*) num ângulo estático Euleriano perfeito livre de ruídos.⁵⁶ No entanto, num paradigma estrito de implantação em TinyML e extração por Inteligência Artificial — onde a topografia central repousa essencialmente em redes neuronais Convolutivas ou Recorrentes capazes de devorar vetores não absolutos numa cadência espectral temporal contínua —, o cálculo das primitivas de rotação com matriz trigonométrica complexa constitui habitualmente fardo sobrelevado.⁵⁸ Frequentemente, atirá-las em formato direto das grandezas vetorizadas atenuadas apenas com o limite de hardware (DLPF) traduz-se no formato ótimo e natural para um treino focado.⁵⁹

5.2. Debouncing Analógico em Resistências Dinâmicas (Flex Sensors)

Na transição de uma luva em repouso passivo até uma falange contraída num evento abrupto mecânico, as propriedades cinemáticas intrínsecas da membrana semicondutora carbonada que envolve cada *Flex Sensor* variam violentamente a sua condutância. Durante o fechar instável em volta de si mesma, a resistência imposta no divisor de tensão cria picos espasmódicos (*bouncing* errático análogo à turbulência mecânica descrita na instabilidade interativa de botões simples do ambiente Arduino) na captação por via do circuito da ponte ADC do STM32.⁶⁰

De forma a uniformizar o perfil da matriz elétrica devolvida, seria preferível integrar o algoritmo de Suavização em *Janela Variável Exponencial* (Filtro EMA – *Exponential Moving Average*), processado localmente imediatamente em trânsito sucessivo à emissão central despoletada via registo automático (ISR / DMA).⁶¹ Contudo, processar as divisões associadas a ponto flutuante ($S_n = \alpha \cdot X_n + (1 - \alpha) \cdot S_{n-1}$) acorrentaria a estabilidade determinística e latente dos ciclos exíguos de um microcontrolador em rotinas críticas de ISR ou varrimento rápido contínuo.⁶²

A conversão metodológica testada incansavelmente sobre plataformas integradas traduz esta filtragem da classe matemática das frações e decimais diretos (*Float*) numa operação de arquitetura assente unicamente em manipulação *Bitwise* Estática (inteiros por deslocamento lógicos >> da camada assembly original).

O paradigma do código formatado implementável diretamente para a filtragem contínua via janela do canal do MCU na forma otimizada exhibe:

C

```
// Instalação do array dinâmico com acúmulos progressivos temporais
uint32_t medias_historicas_flex = {0};

// Constantes estáticas matemáticas.
// O Fator 3 infere divisão logarítmica absoluta pelo deslocamento de bits (2^3 = 8).
const unsigned int expoente_gravidade_ema = 3;
const unsigned int variavel_escala_mascara = (1U << expoente_gravidade_ema) - 1U;

// Rotina leve de processamento para invocação síncrona nos fluxos analógicos lidos
void integrar_filtro_EMA_bitwise(uint16_t* vetor_sinais_em_bruto) {
    for (int posicao = 0; posicao < NUM_CANAL_FLEX_ADC; posicao++) {
        // Multiplicação por 7 (escala) somado com o sinal de entrada bruto
        // e divido inteiramente por deslocamento global binário à direita.
        medias_historicas_flex[posicao] =
            (medias_historicas_flex[posicao] * variavel_escala_mascara +
             vetor_sinais_em_bruto[posicao])
            >> expoente_gravidade_ema;

        // Reescreve in loco a amostra suavizada sem inflar RAM adicional.
        vetor_sinais_em_bruto[posicao] = (uint16_t)medias_historicas_flex[posicao];
    }
}
```

O retardo micro-segundal inserido iterativamente em fase logarítmica por tal algoritmo amortece de forma sublime picos vibracionais elétricos, concedendo aos fechos biológicos em transição um arco progressivo e perfeitamente harmónico para as avaliações lineares inter-nós da camada em TinyML, impedindo categoricamente corrupções e ambiguidades que assombram *features* e sobrepõem transições sensoriais desequilibradas.⁶²

6. Estruturação do Firmware em Fase de Inferência Inteligente

A consolidação da abstração em hardware liberta o caminho para o núcleo do propósito final do STM32F411RE: não ser apenas um mero relé passivo gravador de dados sensíveis na via paralela externa, mas converter-se na totalidade isolada de processamento de topologia das máquinas de *Edge-Computing*. O objetivo dita que a placa processará uma inferência TinyML autónoma sobre o pacote em contínuo armazenamento temporal (de 50Hz em pacotes retidos na ordem de um número definido de matrizes), deduzindo estritamente as previsões do seu peso sináptico embarcado. A exfiltração resultante da arquitetura consistirá *exclusivamente*

numa String compacta portadora da classificação e confiança final, despachada num envelope mínimo sem-fios que abaterá consideravelmente o consumo energético desastroso gerado na manutenção ativa de grandes blocos de rádio frequência num HM-10 ou no clássico módulo HC-05.⁶³

6.1. Otimização do Mapeamento do Cérebro Embutido

A exportação pré-validada do C++ CUBEMX gerada via exportações da plataforma parceira Edge Impulse/ X-Cube-AI manifesta exigências rigorosas associadas ao rácio volumétrico e operacional.¹³ O STM32 restrito possui 128K SRAM. Se formos complacentes, apenas a ativação dos pacotes binários flutuantes (FP32) do tensor primário causará uma esmagadora panóplia em colisão direta com erros lógicos por restrição fatal do endereçamento e limites transbordados em falha global do firmware.⁶

A engenharia deve assumir que os pesos fixos (rom estática interativa que consome o patamar restrito do *Flash*) devem ser combinados obrigatoriamente sob uma malha profunda de *Quantização Estática Inteira (INT-8)*.²⁶ Este método de escalagem restringe os patamares da vírgula flutuante complexa original entre um intervalo exíguo em dígitos de limites formatados, condensando a totalidade das camadas do motor em modelos altamente viáveis sob perdas teóricas inferenciais reduzidas e inócuas para os movimentos balísticos gerais mapeados nos gestos amplos do homem.²⁴ Em adição, a compilação paralela impõe que definições flutuantes subjacentes (presente nas bibliotecas robustas e padronizadas no GNU, relativas à família explícita de `printf()`) devem ser isoladas, retiradas da máscara interativa e suspensas nas *flags* internas operativas (`-u_printf_float`), o que devolverá e estilizará uma massiva sobrecarga de alocação redundante superior à gama crítica associada aos tensores ativados do código do TinyML em ambiente ST.⁶⁴

6.2. Ciclo Deslizante de Inferência e Máquina Lógica (Sliding Window)

Num regime passivo da luva inteligente, a deteção em tempo real pressupõe um processo cíclico focado numa Janela Temporal de Avaliação Constante e Recorrente, classificada arquiteturalmente na categoria técnica de Janela Móvel Deslizante (*Sliding Window*). Operando na estabilidade do intervalo de ciclo definido a 50Hz, um período padrão classificado na demarcação cinemática restrita de 1 segundo de tempo humano produzirá no imediato 50 pontos focais únicos na matriz.¹⁹ Contendo a matriz agregada a uma grelha contendo 11 vetores em trânsito contínuo da anatomia física interativa, isto traduz-se no preenchimento unidimensional exigido aos conectores da primeira malha lógica da rede (uma rede vetorial estrutural em tensor intermédio equivalente aos 550 blocos base para interpretação global dos pesos neuronais).¹⁹

No entanto, as pessoas não se movem em espartilhos segmentados no intervalo perfeito assinalado de um relógio base. Um corte rigoroso e arbitrário entre pacotes sucessivos pode fraturar completamente um punho gestual em colisão nos domínios interligados e cruzados

que ligam os dois momentos assíncronos das margens dos segundos definidos.

Para preservar a fluidez ininterrupta orgânica que define a naturalidade da classificação em real-time do TinyML, a janela tem de ser instanciada como um formato contínuo e sobreponível. Em vez de suspender, purgar, limpar e recolher de raiz a totalidade de zero os 50 dados paralelos das avaliações do ADC para instigar inferência global (gerando vácuos estáticos), as chamadas matemáticas processam-se no topo da atualização sobre *Stride*. Os registos do DMA varrem iterativamente a base passiva rotineira das matrizes no fundo de escala circulante dos anéis (por exemplo o buffer expulsa as 25 unidades passivas distantes antigas temporais e enlaça mecanicamente os vetores recém atualizados, forçando uma repetição logarítmica com a sobreposição interativa da matriz a 50%).

Quando este vetor consolidado alcança a fronteira estruturada de bloqueio processual na sua sobreposição cíclica estrita no *Main While Loop*:

1. Os 550 nós matriciais em bruto passam pelo processo matemático local gerado via Cube.MX focado no DSP central da Edge Impulse (Análise via FFT transformadas, limitação transversal ou análise paralela espectral se a plataforma o induzir).¹⁹
2. O API injetável nativo extraído da malha analítica é exposta via um ponteiro processado pelo comando imperativo de chamamento ao tensor oculto classificado na HAL STM32 gerado nas portas centrais (designações explícitas nativas como funções de invocação da raiz processual `ai_network_run()`).⁶⁵
3. O Cortex-M4 desvia a exclusividade da estrutura principal, computa a densidade iterativa multiplicando massivamente vetores e inter-sinapses (em patamares milissegundos) e exibe em tabela de classes em retorno final na memória a atribuição global quantificável dos limites paralelos preditivos da classificação, e probabilidades inerentes individuais em formato decimal estrito (*Confidence Score*) gerados.
4. É induzido um *Limiar de Decisão Logístico (Threshold)*: Se as inferências em conflito gerarem para a designação provável prioritária um nível que gravite em confianças não esmagadoras (por exemplo, abaixo da fronteira interativa da marca delimitada nos estatutos operacionais de segurança avaliados perto dos rígidos 85%), o processador rejeita passivamente, silencia o módulo de emissões RF exterior e decreta formalmente na rotina isolada a condição base em limbo transitório nulo.
5. Sendo confirmada uma decisão probabilística superior isolada, a *string* exata minimalizada do gesto é expedida para a UART ativa apontada, transmitindo finalmente uma fração de memória compacta (como "Gesto_Letra: L \n") aos recetores emparelhados remotos via comunicações por radiofrequência sem-fios.⁶³ A drástica substituição no tráfego maciço via ar (antes encharcado com séries temporais dos onze vetores em bruto contínuos expostos exaustivamente em cadência perene de tráfego analógico maciço de PC) restringe a uma poupança esmagadora das correntes limite imposta pela fragilidade global e viabilidade energética do empacotamento prático e real para uso diário na base das matrizes ligadas em formato TinyML IoT portátil.⁶³

7. Apresentação Remota e o *Dashboard* Local da Interface (Web GUI)

Consumados e finalizados internamente o processo inferencial da placa estrita isolada conectada na Luva (o Edge) via processadores, o nó secundário atua sobre a exibição no ecrã remoto terminal num PC via comunicações de rádio paralela e módulos agregados. Sendo que a arquitetura central base de Hardware dispôs, pelas condicionantes do fabricante, do HM-10 (BLE) ou alternativamente no mais antiquado base SPP Classic de comunicação cruzada HC-05 conectado nativamente aos barramentos assíncronos USART, o PC terá obrigatoriamente de simular um emparelhamento clássico (Bluetooth para o HC-05 simula rotineiramente e perfeitamente um porto Virtual de Série genérica nas matrizes operativas de SO anfitrião do PC em paralelo com tramas normais USB sem intervenção profunda instaladora ou bloqueios complexos).

O planeamento estrito da aplicação que visualiza os ritmos exatos do reconhecimento e devolve uma *User Interface (UI)* apelativa e instantânea na central base de ecrã necessita de uma plataforma *Full-Stack* passiva desprovida de atritos interativos complexos ou dependências que sufoquem o ritmo nativo.¹² O mercado debate as ferramentas mais otimizadas.⁶⁶

7.1. Análise Focada das Tecnologias Paralelas Existentes (*Stacks*)

1. **A Web Bluetooth API:** A recente expansão interativa do painel API focada estritamente na plataforma em navegadores motores de núcleo base e ecossistema local do consórcio Chromium permitiria de base, um cenário puramente *front-end*, emparelhando um *layout HTML/JS* estéril estritamente com os protocolos e atributos GATT da norma sem intermédio pesado ou software local secundário injetado como servidor cruzado da base PC.⁶⁷ A eliminação teórica formidável e tentadora de latências seria inegável. Não obstante, o protocolo e norma subjacente focado imperativamente baseia a funcionalidade transversal num imperativo inultrapassável: O chip integrado transmissor tem obrigatoriamente de emitir numa modulação suportada em Low Energy (BLE), a norma da família paralela 4.0 superior. Dada as características obsoletas do transmissor HC-05 do projeto (suportado nas especificações clássicas Bluetooth 2.0 Base EDR Serial Port Profile)⁶⁹, a conectividade central inter-camadas com *Web Bluetooth API* assume incompatibilidade nativa e repulsa imperativa. Relega-se para obsolescência inviável para o cenário delineado na premissa e impossibilita implementação real.
2. **Python c/ Microframework Flask & Módulo Assíncrono:** O vasto conhecimento comunitário de inteligência virtual atira em primeira mão para as soluções focadas nas vertentes limpas em micro-estruturas e domínios da linguagem *Python*. Um código com a rotina principal instanciada paralelamente no pacote PySerial consegue acoplar-se formidavelmente à escuta permanente das rotinas COM emparelhadas pelo hardware base exterior do HC-05.⁴⁹ Contudo, a apresentação visual remete a construção aos canais

WSGI (síncronos baseados num bloqueio pesado do paradigma subjacente a operações passivas I/O) originados nativamente no Flask perante tráfego de chegada em série.⁷¹ Para gerar emissores constantes diretos num *Socket* na folha HTML terminal desprovidos de atrito (*Polling REST constante exíguo*) ter-se-ia obrigatoriamente de inserir ou transitar uma camada de acoplagem massivamente complexa, pesada em *overhead*, que reencaminha e interceta blocos e rotas no núcleo do Flask-SocketIO e *Server-Sent Events* assíncronos sobrepostas em ferramentas de manipulação de blocos parciais virtuais *Gevent*.⁷¹ Tais construções forçadas são famigeradas nas análises de latência empírica das rotinas e de arquitetura processual a incorrer em quebras rítmicas indesejadas, estrangulamentos do GIL (Global Interpreter Lock nativo e castrador intermitente transversal associado ao motor compilador original do *Python* base nas suas instâncias C).⁷¹

3. **Arquitetura Assíncrona Nativa: Node.js + Modulação de Eventos em Socket.io:** No vértice diametralmente oposto da latência obstrutiva assenta o motor massivo de execução escalável exterior desenvolvido pelo Chrome (O modelo em motor compilação extrema em just-in-time denominado de *V8 JavaScript Engine* que alimenta a abstração robusta processual global da esfera *Node.js*). Esta escolha tecnológica liberta o PC de amarras bloqueantes por desenho base.⁷³ Em cenários práticos, o emparelhamento não-síncrono e *event-driven loop* processual interno construído pelas ferramentas base processuais geradoras *libuv* engole em tempo sub-milissegundal todos os despoletadores sem bloqueios latentes gerados ao largo das transferências I/O incessantes e densas da serial, convertendo *Node* na espinha dorsal recomendada indiscutível neste paradigma do modelo proposto.⁷³ Adicionalmente, quando acoplado à matriz e ferramentas robustas e leves como a bibliotecas abertas das pontes paralizadas assíncronas *serialport* gerida nativamente com o padrão da norma clássica mundial de túneis abertos síncronos da internet moderna assentes nas conexões *Websockets* estritamente atenuadas providenciado globalmente com interações leves no ecossistema subjacente de pacotes de evento base do gestor universal de roteamento e envio interativo do *Socket.io*, constitui a derradeira solução.⁷⁷

7.2. Codificação Proposta Transversal e Otimizada em Node.js

O trâmite operacional da fundação do gestor assume uma implementação ligeira da biblioteca modular *Express* servindo os ficheiros vitais UI num servidor restrito interno, simultaneamente monitorizando via *stream* contínuo ativado as *ComPorts* ativas:

JavaScript

```
// Importações modulares nucleares para arquitetura de baixa latência
const aplicacaoExpress = require('express');
```

```

const Modulo_HTTP = require('http');
const { Server } = require("socket.io");
const { SerialPort, ReadlineParser } = require('serialport');

const app = aplicacaoExpress();
const ServidorCentral = Modulo_HTTP.createServer(app);
const Túnel_Websocket = new Server(ServidorCentral);

// Abstração da diretoria base onde jaz o formato estético index.html
app.use(aplicacaoExpress.static('public'));

/* Ligação com o driver subjacente na abstração transversal da porta paralela no sistema anfitrião
(O sistema de emparelhamento do HC-05 no OS originou, a título de exemplo referencial iterativo,
uma instância ativa na COM5 com o padrão predefinido a 115200 de velocidade baud nativa). */
const InterfaceHardware_HC05 = new SerialPort({ path: 'COM5', baudRate: 115200 });

// Formataador e filtro assíncrono que acumula o ruído numérico intermédio das tramitações atômicas da
USB/Serial,
// gerando alertas na pool V8 estritamente e organicamente, assim que acarreta no desvio explícito
condicionado ao formato limpo terminal (\n)
const MotorExtrator_Delimitador_Linha = InterfaceHardware_HC05.pipe(new ReadlineParser({
delimiter: '\n' }));

// Gatilho imperativo na pool Event Loop base do NodeJS de escuta não obstrutiva da COM.
MotorExtrator_Delimitador_Linha.on('data', (dado_formatado_recebido) => {
  // Log interno base para verificação técnica processual
  console.log(`[+] Fluxo TinyML: O Reconhecimento inferido aponta como:
${dado_formatado_recebido}`);

  /* Emissão global interativa em JSON instantâneo nos pacotes de WebSockets a atuar sobre matrizes
ativas.
Difunde nativamente na rede a cada milissegundo as strings tratadas do edge-board sem delays
obstrutivos no sistema */
  Túnel_Websocket.emit('novo_reconhecimento_concluido', { classificação:
dado_formatado_recebido.trim() });
});

ServidorCentral.listen(3000, () => {
  console.log('Painel de Vigilância HMI Local Invocado no Endereço Restrito Local:
http://localhost:3000');
});

```

Na folha formatada limpa renderizável assente nas instâncias processuais que flui no navegador final base gerada na linguagem estática interativa e flexível clássica global orientada

a *DOM* do Vanilla JS, a instrução `socket.on('novo_reconhecimento_concluido')` garante que os textos base com os atributos visuais no centro de gravidade da aba ou folha de base UI intercedem instantaneamente na grelha HTML através de simples abstrações puras como `document.getElementById` com injeção base `innerText`, materializando visualmente sem sobressaltos (*refresh rate obstrutivo* anulado), numa topologia sem latências pesadas as comunicações inferenciais originadas pelo TinyML embarcado à exígua fração temporal de distância real.¹²

8. Abordagens Estratégicas Comprovadas em Repositórios Comunitários (GitHub/StackOverflow)

A validação de implementações complexas e cruzadas da orquestração sensível e exigente originada ao incorporar redes *Deep Learning* restritas sob hardware 32-bits embarcado baseia-se muitas vezes na análise comparativa de soluções arquiteturais já sedimentadas em pacotes *open-source* globais. A exploração cruzada nos ecossistemas e fóruns subjacentes às matrizes ativas fornece perspectivas únicas e *workarounds* em C++ ou em fluxos interativos da ST para a ultrapassagem de obstáculos clássicos.

- **teomaras76/AI_on_STM32_Gesture_recognition:** Este projeto corporiza uma abstração cabal sobre o paradigma de processamento nativo *offline* nas instâncias paralelas do ecossistema de avaliação do núcleo microcontrolador e placa embarcada sensorial *Sensortile* gerida pelas lógicas interativas intrínsecas focadas nativamente pela própria infraestrutura comercial base da arquitetura da construtora *STMicroelectronics*.⁸⁰ O valioso contributo deste espaço aberto assenta na documentação descritiva fundamental transversal ao porte logístico profundo operado sobre uma topologia global formatada em rede clássica neuronal de modelos em *Keras* nativos para os limiares de código estático C cruzado validando as conversões e quantizações de limites impostas pela ferramenta vitalícia interativa e otimizada da plataforma original nativa *STM32CubeMX.AI* em microcontroladores embarcados paralelos restritos perante a instabilidade estrita e latência na operação e limitações rigorosas no emparelhamento da amostragem em paralelo com os pesados tensores convolucionais processados debaixo de baixos consumos na família *STM32*.⁸⁰
- **devnithw/gesture-tinyml:** Este modelo exposto globalmente dedica-se exhaustivamente ao reconhecimento inferencial com o emprego estrito vetorial na recolha das assinaturas físicas de eixos flutuantes recolhidos num limiar ativo via giroscópios no formato de I2C interativo com sensores como MPU6050, focando as suas *labels* e topologias matriciais no espectro prático analítico restrito de uma luva a induzir classes dicotómicas entre golpes interativos físicos amplos classificados como flexões musculares (*flex*) e agressões balísticas contundentes (*punch*).⁸¹ É exhaustivamente referenciado pelo rigor analítico processado a longo termo e cruzado nos limiares na nuvem sob processadores colaborativos isolados (Google Colab), expondo a metodologia de amostragem matricial das dinâmicas formatadas baseadas em registos isolados síncronos extraíveis na sua

cadência de recolha em rotina para csv com a utilidade e integração fundamental acoplada para conversão cruzada com as metodologias de extração dos formatos flutuantes de *Tensorflow Lite for Microcontrollers (TFLite Micro)*.⁸¹

- **ghimireadarsh/Smart-Glove:** Demonstração complexa integralmente baseada e focada inteiramente na integração restrita focada num tradutor analítico integral focado nas bases semânticas universais estritas englobadas no dialecto cinemático das interações comunicativas humanas padronizadas nativamente para linguagens de sinal gestual, incorporando massivamente os módulos em rede integrados nas matrizes espaciais físicas sobrepostas num limiar tátil contínuo transversal com a extração de eixos fundidos no *accelerómetro/giroscópio* de malha contínua integrados paralelamente numa cadeia combinatória e intersetada via resistências ativas com a amostragem paralela de captação vetorial síncrona gerada em transição pelos canais lógicos flutuantes processados através do conjunto distribuído via multiplicadores analógicos contínuos integrados nos tensores com matrizes dos *Sensores Dinâmicos Físicos Baseados em Membranas Resistivas de Flexão*.⁸³ Aborda e exemplifica metodologias rigorosas baseadas essencialmente com a extração estatística global assente em *Random Forest Classifier* sobre plataformas físicas integradas localizadas na infraestrutura em processadores analíticos globais estritos em malhas sem fios. O repósitório espelha com assertividade a integração cruzada sensorial exigida numa ponte intersetorial de eixos mecânicos no processador central paralelo com emissões estritas na plataforma analítica HMI no emparelhamento via matrizes baseadas nas emissões com interfaces de áudio vocalizados em matrizes traduzidas contínuas textuais formatadas.⁸³
- **UsefulElectronics/stm32-soft-debounce-adc-buttons-read:** Referência empírica transversal na eliminação cirúrgica passiva na modelagem em *software* subjacente sem paralisar e encharcar as interações de atraso ativo do relógio no MCU de anomalias ruidosas provenientes da interação dos canais oscilantes elétricos nas interrupções dinâmicas ativas estritas de pinos (conhecidos por EXTI em abordagens analíticas clássicas em registos de STM) causadas pelo atrito paralelo e caótico impulsionado pelo fenómeno estrito do *bouncing* indesejável elétrico. Mostra como o processamento passivo mitigador sem recurso a abstrações micro-bloqueantes do sistema assente em verificações e varrimentos lógicos e limpos geridos em registo passivo de limites e *timers* paralelos mitiga, enquadra e isola estritamente na perfeição o emparelhamento contínuo dos bloqueios contínuos originados no ruído oscilante nas primitivas dos transistores da unidade.⁸⁴ Fornece um olhar fulcral ao demonstrar de forma taxativa as abstrações interativas base operacionais focadas na extração dos limites com alocação e recolha analógica vetorial gerida puramente e orquestrada debaixo do canal autónomo de base passiva processado no *Direct Memory Access (DMA)* com os recursos lógicos dos ADC interativos, com abordagens essenciais focadas num modelo estruturado na viabilização total de limiar estático perante interações sem as fixações destrutivas impostas pelas restrições na computação imperativa dos processadores base num escalonamento ativo isolado subjacente à sobrecarga da topologia interativa nos ciclos vitais estritos isolados requeridos imperativamente pela cadência crítica restrita associada à extração analítica na

Trabalhos citados

1. MEMS Devices-Based Hand Gesture Recognition via Wearable Computing - MDPI, acesso a março 28, 2026, <https://www.mdpi.com/2072-666X/14/5/947>
2. Smart Glove - GitHub, acesso a março 28, 2026, <https://cepdnack.github.io/e18-3yp-smart-glove/>
3. Camera Control System Based on TinyML Gesture Recognition - IJET-International Journal of Engineering and Technology, acesso a março 28, 2026, <https://www.ijetch.org/2025/IJET-V17N3-1324.pdf>
4. STM32F411RE (128k RAM. 512k Flash) — PlatformIO v6.1 documentation, acesso a março 28, 2026, <https://docs.platformio.org/en/stable/boards/ststm32/genericSTM32F411RE.html>
5. GitHub - trebisky/stm32f411: Bare metal STM32F411 (black pill) programming, acesso a março 28, 2026, <https://github.com/trebisky/stm32f411>
6. Storing huge amount of data on STM32 : r/embedded - Reddit, acesso a março 28, 2026, https://www.reddit.com/r/embedded/comments/1h9hu1x/storing_huge_amount_of_data_on_stm32/
7. MPU-6050 in Action: Practical Guide to Setup, Configuration, and Noise Management, acesso a março 28, 2026, <https://www.allelcoelec.com/blog/mpu-6050-in-action-practical-guide-to-setup-configuration-and-noise-management.html>
8. Pixels-HU/Smart-Glove - GitHub, acesso a março 28, 2026, <https://github.com/Pixels-HU/Smart-Glove>
9. (PDF) Smart Glove - ResearchGate, acesso a março 28, 2026, https://www.researchgate.net/publication/330497137_Smart_Glove
10. Comparison of the functionality HC-05 and HM-10 wireless Bluetooth Modules, acesso a março 28, 2026, <https://plcgoods.com/comparison-of-functionality-hc-05-and-hm-10-wireless-bluetooth-modules/>
11. UART Communication Between STM32F407G and ESP8266: A Practical Guide - Medium, acesso a março 28, 2026, <https://medium.com/@bettaieb88walid/uart-communication-between-stm32f407g-and-esp8266-a-practical-guide-9ac3eb7511b5>
12. Build a Real-Time IoT Environmental Dashboard with Python and Javascript - George Witt, acesso a março 28, 2026, <https://wittgeo.medium.com/build-a-real-time-iot-environmental-dashboard-with-python-and-javascript-715fc07d0c95>
13. Top 8 TinyML Frameworks and Compatible Hardware Platforms (TensorFlow Lite, Edge Impulse, PyTorch Mobile, etc.) - DFRobot, acesso a março 28, 2026, <https://www.dfrobot.com/blog-13921.html>
14. From Tiny Machine Learning to Tiny Deep Learning: A Survey - arXiv, acesso a março 28, 2026, <https://arxiv.org/html/2506.18927v1>

15. Machine Learning on Embedded (Part 4) - Stupid Projects, acesso a março 28, 2026,
<https://www.stupid-projects.com/posts/machine-learning-on-embedded-part-4/>
16. TinyML: Getting Started with TensorFlow Lite for Microcontrollers - DigiKey, acesso a março 28, 2026,
<https://www.digikey.com/en/maker/projects/tinyml-getting-started-with-tensorflow-lite-for-microcontrollers/c0cdd850f5004b098d263400aa294023>
17. TinyML at the Edge: 5 Frameworks Powering IoT | by Nexumo - Medium, acesso a março 28, 2026,
https://medium.com/@Nexumo_/tinyml-at-the-edge-5-frameworks-powering-iot-5c19a78e5211
18. Machine Learning for all STM32 developers with STM32Cube.AI and Edge Impulse, acesso a março 28, 2026,
<https://www.edgeimpulse.com/blog/machine-learning-for-all-stm32-developers-with-stm32cube-ai-and-edge-impulse/>
19. Edge Impulse Data Collection and Training | SiliconWit, acesso a março 28, 2026,
<https://siliconwit.com/education/edge-ai-tinyml/edge-impulse-data-collection-training/>
20. GitHub - jaredmaks/tinyml-on-the-edge: IMU sensors on Arduino Nano 33 BLE Sense can be used in gesture recognition. Using machine learning, a model is trained using the data collected from the IMU sensors on Google Colaboratory. A tensorflow lite model is encoded in arduino header which a classifier sketch uses. They include; accelerometer , gyroscope and magnetometer., acesso a março 28, 2026, <https://github.com/jaredmaks/tinyml-on-the-edge>
21. TinyML Gesture Recognition Tutorial: Building Smart Gesture-Controlled Devices, acesso a março 28, 2026,
<https://thinkrobotics.com/blogs/learn/tinyml-gesture-recognition-tutorial-building-smart-gesture-controlled-devices>
22. Getting started with X-CUBE-AI Expansion Package for Artificial Intelligence (AI), acesso a março 28, 2026,
https://www.stmcu.com.cn/upload/pdf_html/097c2341b5ddf17fb20a99c655069fc2.html
23. UM2526 - Getting started with X-CUBE-AI Expansion Package for Artificial Intelligence (AI) - User manual - STMicroelectronics, acesso a março 28, 2026,
https://www.st.com/resource/en/user_manual/um2526-getting-started-with-xcube-ai-expansion-package-for-artificial-intelligence-ai-stmicroelectronics.pdf
24. AI:STM32Cube.AI model performances - stm32mcu - ST wiki, acesso a março 28, 2026, https://wiki.st.com/stm32mcu/wiki/AI:STM32Cube.AI_model_performances
25. Make Deep Learning Models Run Fast on Embedded Hardware - Edge Impulse, acesso a março 28, 2026,
<https://www.edgeimpulse.com/blog/make-deep-learning-models-run-fast-on-embedded-hardware/>
26. 7 Tips for Optimizing AI Models for Tiny Devices - Edge Impulse, acesso a março 28, 2026,
<https://www.edgeimpulse.com/blog/7-tips-for-optimizing-ai-models-for-tiny-de>

[vices/](#)

27. Edge Impulse Data Collection and Training | SiliconWit, acesso a março 28, 2026, <https://siliconwit.com/education/edge-ai-tinyml/edge-impulse-data-collection-training>
28. IMU Sensor-Based Hand Gesture Recognition for Human-Machine Interfaces - PMC, acesso a março 28, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC6767360/>
29. Influence of Sampling Rate on Wearable IMU Orientation Estimation Accuracy for Human Movement Analysis - PMC, acesso a março 28, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11991382/>
30. Effects of Sampling Frequency on Human Activity Recognition with Machine Learning Aiming at Clinical Applications - MDPI, acesso a março 28, 2026, <https://www.mdpi.com/1424-8220/25/12/3780>
31. Effects of Sampling Frequency on Human Activity Recognition with Machine Learning Aiming at Clinical Applications - PMC, acesso a março 28, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12196717/>
32. Sign Language Recognition Using Wearable Electronics: Implementing k-Nearest Neighbors with Dynamic Time Warping and Convolutional Neural Network Algorithms - MDPI, acesso a março 28, 2026, <https://www.mdpi.com/1424-8220/20/14/3879>
33. Gesture Recognition: TinyML for 8-bit Microcontroller - Hackster.io, acesso a março 28, 2026, <https://www.hackster.io/leonardocavagnis/gesture-recognition-tinyml-for-8-bit-microcontroller-1cb0a8>
34. Hand Gesture Recognition Using Single Patchable Six-Axis Inertial Measurement Unit via Recurrent Neural Networks - PMC, acesso a março 28, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7922880/>
35. Interrupts vs. Polling in Embedded Systems: ESP32 & STM32 Deep Dive | by Sai-2025, acesso a março 28, 2026, <https://sai2025.medium.com/interrupts-vs-polling-in-embedded-systems-esp32-stm32-deep-dive-1e90f05cf0be>
36. STM32 ADC Multiple Channels Using LL Drivers (DMA Mode) - ControllersTech, acesso a março 28, 2026, <https://controllerstech.com/stm32-adc-dma-multiple-channels-ll/>
37. STM32 ADC Application Part 4: Read Multiple Channels Using Interrupt and DMA, acesso a março 28, 2026, <https://blog.embeddedexpert.io/?p=3481>
38. STM32 ADC Timer Trigger Example & External Trigger Sources, acesso a março 28, 2026, <https://deepbluembedded.com/stm32-adc-timer-trigger-example-external-trigger-sources/>
39. Getting Started with STM32G0 and STM32CubeIDE: Multi-channel ADC with DMA and Timer - EmbeddedExpertIO, acesso a março 28, 2026, <https://blog.embeddedexpert.io/?p=2458>
40. Working with ADC and STM32 part 4: Multi Channel Continuous Conversion with DMA, acesso a março 28, 2026, <https://blog.embeddedexpert.io/?p=319>

41. shaunaks-tech/stm32smart: stm32 smart glasses project - GitHub, acesso a março 28, 2026, <https://github.com/shaunaks-tech/stm32smart>
42. STM32 I2C Tutorial HAL Code Examples Slave & Master - DMA / Interrupt, acesso a março 28, 2026, <https://deepbluembedded.com/stm32-i2c-tutorial-hal-examples-slave-dma/>
43. Using STM32 DMA and I2C to read data from MPU6050, acesso a março 28, 2026, https://eggelectricunicycle.bitbucket.io/MicroWorks_30B4_board--Datasheets_30B4--MPU6050--DMA.html
44. GitHub - MarcoMiglio/stm32_mpu6050: MPU6050 (GY-521) Driver for STM32 with Motion and Zero-Motion Detection, acesso a março 28, 2026, https://github.com/MarcoMiglio/stm32_mpu6050
45. STM 32 Streaming ADC Data continuously through UART - EEVblog, acesso a março 28, 2026, <https://www.eevblog.com/forum/projects/stm-32-streaming-adc-data-continuously-through-uart/>
46. Identifying viable UART modules for STM32F411RE via documentation? - Reddit, acesso a março 28, 2026, https://www.reddit.com/r/embedded/comments/1m8i2cl/identifying_viable_uart_modules_for_stm32f411re/
47. STM32 Serial Print & Monitor (UART Data Debug) - DeepBlueMbedded, acesso a março 28, 2026, <https://deepbluembedded.com/stm32-debugging-with-uart-serial-print/>
48. STM32F411 uart asynch - STMicroelectronics Community, acesso a março 28, 2026, <https://community.st.com/t5/stm32-mcus-products/stm32f411-uart-asynch/td-p/472317>
49. Recording Data on a Computer to a CSV File using Python and PySerial, acesso a março 28, 2026, <https://datacapturecontrol.com/articles/data-acquisition/computer-software/pyserial-csv-file>
50. Collecting data from serial port and CSV handling - defgw - WordPress.com, acesso a março 28, 2026, <https://defgw.wordpress.com/2015/04/05/collecting-data-from-serial-port-and-csv-handling/>
51. 16. stm32 UART Driver - Python data logger to CSV File. part 2 - YouTube, acesso a março 28, 2026, <https://www.youtube.com/watch?v=SnpOibgPcss>
52. Working with STM32 and sensors: MPU6050 - EmbeddedExpertIO, acesso a março 28, 2026, <https://blog.embeddedexpert.io/?p=1799>
53. What should i do to eliminate the signal error of MPU 6050 sensor generated by system's vibration : r/arduino - Reddit, acesso a março 28, 2026, https://www.reddit.com/r/arduino/comments/6c2a3t/what_should_i_do_to_eliminate_the_signal_error_of/
54. Problem reading MPU-6050 sensor values with STM32F103RB via I2C, acesso a março 28, 2026, <https://community.st.com/t5/others-stm32-mcus-related/problem-reading-mpu->

- [6050-sensor-values-with-stm32f103rb-via-i2c/td-p/843056](#)
55. Q&A #1 - Noise Reduction, Code Comments, MPU-6050 Datasheet & More Viewer Questions - YouTube, acesso a março 28, 2026,
<https://www.youtube.com/watch?v=BZFu4OHw4GM>
 56. MPU-6050 implementing Kalman and complementary filters - Programming - Arduino Forum, acesso a março 28, 2026,
<https://forum.arduino.cc/t/mpu-6050-implementing-kalman-and-complementary-filters/281422>
 57. Kalman filter vs Complementary filter - YouTube, acesso a março 28, 2026,
<https://www.youtube.com/watch?v=StxgM66iljY>
 58. Implementation of Complimentary filter on MPU 6050 - World Scientific News, acesso a março 28, 2026,
<https://worldscientificnews.com/wp-content/uploads/2025/01/WSN-199-2025-218-234.pdf>
 59. Accelerometer Gesture Recognition - SiliconWit, acesso a março 28, 2026,
<https://siliconwit.com/education/edge-ai-tinyml/accelerometer-gesture-recognition/>
 60. Debouncing/delaying/smoothing input data - Programming - Arduino Forum, acesso a março 28, 2026,
<https://forum.arduino.cc/t/debouncing-delaying-smoothing-input-data/1342638>
 61. Smoothing sensor input - helloCircuits! - WordPress.com, acesso a março 28, 2026,
<https://ericjformanteaching.wordpress.com/2013/10/08/smoothing-sensor-input/>
 62. Improve accuracy of integer multiplication and division for exponential moving average, acesso a março 28, 2026,
<https://stackoverflow.com/questions/77599469/improve-accuracy-of-integer-multiplication-and-division-for-exponential-moving-a>
 63. Benchmarking latency across common wireless links for microcontrollers - Electric UI, acesso a março 28, 2026,
<https://electricui.com/blog/latency-comparison>
 64. TinyML: Getting Started with STM32 X-CUBE-AI - DigiKey, acesso a março 28, 2026,
<https://www.digikey.com/en/maker/projects/tinyml-getting-started-with-stm32-x-cube-ai/f94e1c8bfc1e4b6291d0f672d780d2c0>
 65. How to deploy artificial intelligence algorithms on STM32? - Blog - Ampheo, acesso a março 28, 2026,
<https://www.ampheo.com/blog/how-to-deploy-artificial-intelligence-algorithms-on-stm32>
 66. Comparison of Bluetooth Low Energy (BLE), Wi-Fi, Serial and 5G in IoMT - SciTePress, acesso a março 28, 2026,
<https://www.scitepress.org/Papers/2025/133215/133215.pdf>
 67. Using Web Bluetooth To Read BBQ Temperature Sensor Data - PQINA, acesso a março 28, 2026,
<https://pqina.nl/blog/web-bluetooth-api-read-bbq-temperature-sensor-data/>
 68. Web Bluetooth Guide: How to Build a Real-Time Heart Rate Dashboard - DEV

- Community, acesso a março 28, 2026,
<https://dev.to/wellallytech/web-bluetooth-guide-how-to-build-a-real-time-heart-rate-dashboard-nj7>
69. Hc-05 and HM-10 - Networking, Protocols, and Devices - Arduino Forum, acesso a março 28, 2026, <https://forum.arduino.cc/t/hc-05-and-hm-10/1091602>
 70. HM-10 Bluetooth 4 BLE Modules - Martyn Currey, acesso a março 28, 2026, <https://www.martyncurrey.com/hm-10-bluetooth-4ble-modules/>
 71. Python Flask vs Node.js Express. When building web applications... | by Jason Roell | Medium, acesso a março 28, 2026, <https://medium.com/@roelljr/python-flask-vs-node-js-express-4662b6f97b28>
 72. BUILD REAL-TIME LOG-EVENTS DASHBOARD USING FLASK and SERVER-SENT-EVENTS | by Nathaniel Cobbinah | Medium, acesso a março 28, 2026, <https://medium.com/@fmg3ckali/build-real-time-log-events-dashboard-using-flask-and-server-sent-events-cb0b0721c65f>
 73. Node.js vs Python: Real Benchmarks, Performance Insights, and Scalability Analysis, acesso a março 28, 2026, <https://dev.to/m-a-h-b-u-b/nodejs-vs-python-real-benchmarks-performance-insights-and-scalability-analysis-4dm5>
 74. This project compares the performance of simple HTTP servers built with Node.js, Python (Flask), and Go. The comparison measures requests per second, latency, and data transfer rate under concurrent load. · GitHub, acesso a março 28, 2026, <https://github.com/Aatom003/Go-Node-Python>
 75. Flask vs Node.js - Which Backend Framework to Choose in 2026 - Windframe, acesso a março 28, 2026, <https://windframe.dev/blog/flask-vs-nodejs>
 76. Node.js Vs Flask: Which One Has A Better Performance | HackerNoon, acesso a março 28, 2026, <https://hackernoon.com/nodejs-vs-flask-which-one-has-a-better-performance-vf1u35gr>
 77. Developing a Real-time Dashboard with Flask, Postgres, and Socket.IO | TestDriven.io, acesso a março 28, 2026, <https://testdriven.io/blog/flask-postgres-socketio/>
 78. Display Real Time Updates Using Python, Flask and Websocket - YouTube, acesso a março 28, 2026, <https://www.youtube.com/watch?v=dPhr1YCQTU4>
 79. Display real time data from a bluetooth device on a flask website : r/learnpython - Reddit, acesso a março 28, 2026, https://www.reddit.com/r/learnpython/comments/zkannf/display_real_time_data_from_a_bluetooth_device_on/
 80. teomaras76/AI_on_STM32_Gesture_recognition: Gesture recognition Deep Learning project for STM32 (based on Keras model) - GitHub, acesso a março 28, 2026, https://github.com/teomaras76/AI_on_STM32_Gesture_recognition
 81. devnithw/gesture-tinyml: Embedded machine learning for gesture recognition using Arduino Nano 33 BLE Sense. Includes source code for programming the board and notebooks for training the model. - GitHub, acesso a março 28, 2026, <https://github.com/devnithw/gesture-tinyml>

82. Gesture Recognition using TinyML (2022) | Learn Technology in 5 Minutes - YouTube, acesso a março 28, 2026,
<https://www.youtube.com/watch?v=9qqbjpJiGkc>
83. GitHub - ghimireadarsh/Smart-Glove: A final year project of my Bachelor in Electronics and Communication Engineering, acesso a março 28, 2026,
<https://github.com/ghimireadarsh/Smart-Glove>
84. STM32 Access Multiple Buttons Through One Pin Using ADC and DMA With Software Debounce algorithm - GitHub, acesso a março 28, 2026,
<https://github.com/UsefulElectronics/stm32-soft-debounce-adc-buttons-read>
85. stm32-soft-debounce-adc-buttons-read/README.md at main - GitHub, acesso a março 28, 2026,
<https://github.com/UsefulElectronics/stm32-soft-debounce-adc-buttons-read/blob/main/README.md>
86. STM32 Tutorial #50 - BOUNCE and how to DEBOUNCE - YouTube, acesso a março 28, 2026, <https://www.youtube.com/watch?v=dyG6BaKxDIA>
87. STM32 Button Debounce Code Examples [Tutorial] - DeepBlueMbedded, acesso a março 28, 2026,
<https://deepbluembedded.com/stm32-button-debounce-code-examples-tutorial/>